

## 0. Presentación del grupo 7

| Nombre y apellido       | Dirección de correo electrónico  |
|-------------------------|----------------------------------|
| Tomas Dominguez         |                                  |
| Sergio José Martellotti | martellottisergio@hotmail.com.ar |
| Ignacio Correa          | ignaciomanuel07@gmail.com        |

Acceso a diagrama UML: <https://tinyurl.com/tpgrupo7>

Repositorio de GitHub: <https://github.com/IgnaciomCorrea/TP-Integrador-Ecommerce-Obj-2>

### Introducción

El presente trabajo documenta las decisiones de diseño tomadas al modelar el sistema de gestión de pedidos, catálogo y pagos de una plataforma de e-commerce. A lo largo del desarrollo nos encontramos con distintos problemas de diseño recurrentes (comportamientos que varían según un estado, algoritmos intercambiables, estructuras jerárquicas de objetos, notificaciones desacopladas entre módulos) para los cuales aplicamos los patrones de diseño que mejor se ajustaban a cada contexto, priorizando siempre el bajo acoplamiento y el cumplimiento de los principios SOLID.

En primer lugar, para el catálogo de productos se utilizó el patrón **Composite**, ya que el sistema debe representar productos individuales y paquetes que pueden contener otros paquetes. Donde el sistema pueda tratarlos con la misma interfaz.

En segundo lugar, para el ciclo de vida del pedido utilizamos el patrón **State**, dado que el comportamiento del Pedido irá siendo modificado, dependiendo del estado en el que se encuentre.

En tercer lugar, utilizamos un **Template Method** para la validación de los métodos de pago, ya que estos requieren realizar una validación, donde cada método de pago requiere distintos datos y métodos pero donde todos tienen el mismo flujo general de procesamiento: validar, reservar fondos, ejecutar la transacción y notificar. Con el Template Method, definimos la estructura del algoritmo y cada paso se especifica en las subclases.

En cuarto lugar, para los métodos de envío se utilizó el patrón **Strategy**, ya que el método de envío es una elección del cliente. Estos encapsulan cada algoritmo de

cálculo de costo y tiempo de entrega de las estrategias. Estas son intercambiables en tiempo de ejecución.

En quinto lugar, para el sistema de notificaciones utilizamos el patrón **Observer**, con el fin de desacoplar al Pedido de los distintos módulos que deben reaccionar a los cambios del Pedido.

En sexto lugar, para la búsqueda y filtro en el catálogo, se utilizó nuevamente el patrón **Composite**, dado que los criterios de búsqueda pueden ser simples o compuestos. Estos deben ser combinables de forma flexible y recursiva, sin que el cliente deba distinguir en sus clases.

En último lugar, para la generación de reportes se aplicó el patrón **Visitor**, ya que se necesitaba definir una nueva operación (el cálculo de productos más vendidos) sobre la estructura existente de Producto y Paquete sin modificar dichas clases ni acoplarlas a la lógica de reportería. De esta forma, cada tipo de reporte puede implementarse como un nuevo visitor independiente, mientras que la recolección de datos queda completamente separada de su posterior exportación a los distintos formatos de salida (texto plano, CSV, HTML).

# 1. Catálogo de productos

## 1. Contexto

El sistema debe soportar tanto productos simples como compuestos (paquetes), donde incluso los paquetes pueden contener otros paquetes anidados.

## 2. Solución Propuesta: Patrón Composite

El modelado del catálogo de productos fue diseñado con una estructura centrada en el **Patrón de diseño Composite**, con este buscamos que el cliente del catálogo no necesite distinguir entre productos simples y compuestos.

Para el diseño contamos con una **clase abstracta Vendible** que presenta el contrato y la interfaz que expondrán los productos y los paquetes.

- `getPrecioBase(): Double`
- `getPrecioFinal(): Double`
- `getNombre(): String`
- `getDescripcion(): String`
- `getCategoria(): Categoria`
- `validarVendible(): boolean`

De esta se desprenden **dos subclases concretas**:

Por un lado, **Producto** que representa un **elemento individual del catálogo**. Cuenta con los atributos heredados de **Vendible** e implementa los métodos `getPrecioBase` y `getPrecioFinal`. Donde el primero retorna el precio del producto y el segundo aplica el descuento al precio base.

Cuenta además con métodos que agregan instancias de la **clase Atributo** para la manipulación de **atributos dinámicos**. Esta clase **Atributo** contiene dos atributos, un `String` para representar el nombre del Atributo y un atributo valor que es de tipo abstracto.

Por otro lado, tenemos la **clase Paquete** que representa un **grupo de vendibles**, esta cuenta con una conjunto de instancias de **ItemVendible** que describe un **Vendible** y la cantidad de este que hay en paquete. De esta manera podemos representar un paquete de n cantidad de unidades del mismo producto sin la necesidad de repetir el objeto n veces en la lista, y así evitar recorridos de lista o posibles inconsistencias.

**ItemVendible** conoce o sabe calcular su precio multiplicando el `precioBase()` del vendible que tiene multiplicado por la cantidad.

La forma en que **Paquete** calcula su precio, como así lo hace **Producto**, implica dos métodos:

- **getPrecioBase()** que recorre su estructura recursivamente, acumulando los precioBase de sus productos y paquetes.
- **getPrecioFinal()** que aplica el descuento del Paquete sobre el precio base total.

### 3. Asignación de responsabilidades

Con todo esto buscamos cumplir **los principios SOLID**:

- Cada clase tiene una única responsabilidad
  - **Producto** representa un elemento individual.
  - **Paquete** representa un conjunto de productos o de conjuntos.
  - **ItemVendible** permite representar multiples unidades de productos.
  - **Atributo** encapsula un atributo dinámico para Producto.
- Podemos agregar nuevos tipos de Vendibles sin modificar los ya existentes, cumpliendo con el principio Open/Closed.
- Todos los Vendibles pueden tratarse de igual manera, sin necesidad de distinguir si son simples o compuestos.

En lo que respecta al **control de stock**, se diseñó la clase **StockVendible**, que asocia un Vendible con un número entero que representa la cantidad de stock disponible. El **Catalogo** cuenta con una lista de instancias de **StockVendible** donde se registrará el stock disponible y sobre el cual se consultará la disposición de productos para validar ventas.

### 4. Conclusiones

El uso del patrón Composite permite modelar de manera uniforme estructuras simples y compuestas, facilitando la extensibilidad del sistema y reduciendo el acoplamiento entre componentes.

Además, la asignación clara de responsabilidades y el uso de clases auxiliares como ItemVendible y Atributo contribuyen a un diseño más mantenible, coherente y alineado con los principios SOLID.

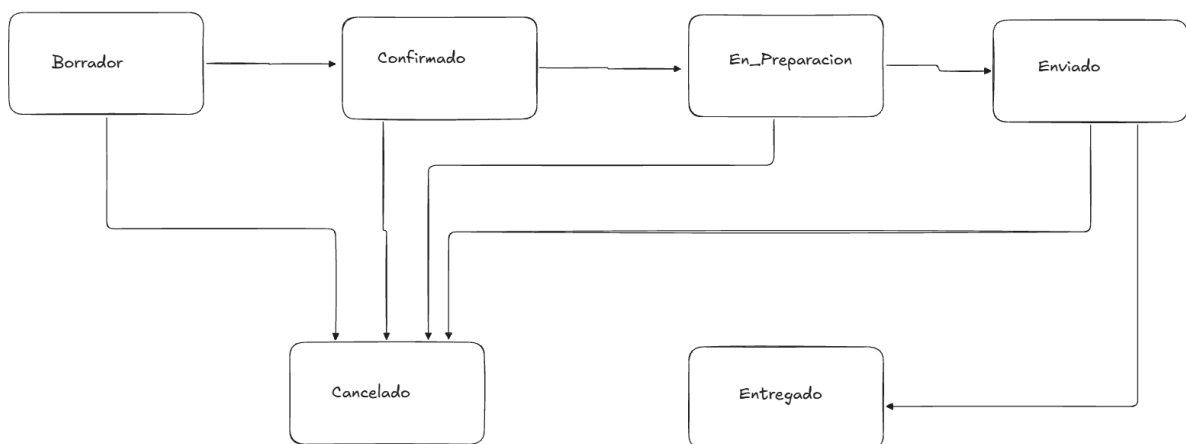
## 2. Ciclo de vida del pedido

### 1. Contexto

Un pedido en el sistema cambia su comportamiento a lo largo del ciclo de vida.

(Borrador → Confirmado → EnPreparación → Enviado → Entregado/Cancelado)

Identificamos este flujo, donde Borrador es el estado inicial de un Pedido y los estados terminales son Cancelado y Entregado.



Dependiendo del estado actual, existen operaciones que son válidas y otras que no. Por ejemplo:

En Borrador se pueden agregar o quitar ítems y pasar a Confirmado o Cancelado, en Confirmado no se puede modificar el carrito pero se puede pasar a EnPreparación o Cancelado, en Enviado al cancelar sólo reembolsa los productos, no el envío, etc.

Si se implementara con condicionales (if/else o switch), el código de la clase Pedido se llenaría de lógica, ya que este debería consultar a su estado interno en cuál se encuentra y tomar una decisión basado en ello.

Esto genera alto acoplamiento, código repetitivo y viola el principio Open/Closed, porque cada nuevo estado que se pueda agregar implica modificar la clase Pedido.

### 2. Solución Propuesta: Patrón State

Optamos por el patrón State ya que permite que un objeto altere su comportamiento interno cuando cambia su estado, sin implementar múltiples condicionales dentro de pedido. Además, la clase conoce una cantidad finita de estados (que en el dominio actual son Borrador, Confirmado, En\_Preparación, Enviado, Cancelado y Entregado).

La estructura que proponemos para esta solución es la siguiente:

Contexto (Pedido): almacena una referencia al estado actual y delega todas las responsabilidades al estado en cuestión.

Estado (EstadoPedido): interfaz que declara las operaciones que pueden variar según el estado que se esté utilizando.

Estados concretos (Borrador, Confirmado, EnPreparacion, etc): implementan la interfaz anteriormente mencionada y el comportamiento específico para cada estado.

### 3. ¿Por qué delegar responsabilidades a los estados concretos?

Hemos optado por la delegación total. Cada estado concreto es responsable de:

- Validar si la operación ejecutada es permitida en ese estado (Ej, agregar productos en Enviado, Cancelar en Entregado, etc).
- Ejecutar la lógica de negocio de otras clases (decrementar stock, generar nota de crédito, etc).
- Decidir a qué estado pasar (utilizando pedido.setEstado(...)).

Ventajas de esta decisión:

Sigue los principios del patrón - Es la implementación más fiel al State explicado en el libro de Gamma et al.

Bajo acoplamiento - La clase Pedido no conoce los estados concretos, solo depende de la interfaz EstadoPedido.

Alta cohesión - Cada estado concentra todo el comportamiento que le corresponde. La lógica de cancelación en Confirmado, EnPreparacion y Enviado está dentro de cada clase, no dispersa en Pedido.

Escalabilidad - Para agregar un nuevo estado (ej. Reembolsado, Extraviado, etc), solo se crea una nueva clase; no se modifica Pedido ni los estados existentes.

Mantenibilidad - El código es más prolijo y menos propenso a errores, debido a la baja cantidad de "instance of <<Estado>>" o condicionales varios dispersos por la clase Pedido.

## 4. ¿Qué responsabilidad tiene Pedido?

La clase Pedido no decide ni ejecuta la lógica de los cambios de estado. Su única responsabilidad es:

- Almacenar el estado actual y la lista de Vendible (también el manejo de notificaciones del punto 2.5).
- Exponer métodos públicos que delegan inmediatamente en el estado actual.
- Proveer un método setEstado para que los estados concretos puedan actualizar la referencia.
- De esta forma, el contexto se mantiene ignorante de las reglas de negocio de cada estado, lo que centraliza la variabilidad en las clases de estado.

## 5. Cumplimiento de conceptos aprendidos:

Respetar el principio SOLID

1. **Single Responsibility:** Pedido solo se ocupa de delegar; cada estado concreto tiene una única razón de cambio.
2. **Open/Close:** El sistema está abierto a agregar nuevos estados y cerrado a modificar Pedido.
3. **Liskov Substitution:** Cualquier estado puede reemplazar a otro sin alterar el correcto funcionamiento de Pedido.
4. **Interface Segregation:** La interfaz <<EstadoPedido>> declara sólo los métodos que el contexto necesita.
5. **Dependency inversion:** Pedido depende de la abstracción <<EstadoPedido>>, no de implementaciones concretas.

## 3. Métodos de envío

### 1. Contexto y problema

El sistema debe soportar tres métodos de envío, cada uno con su propia lógica para calcular el costo y el tiempo de entrega:

Envío estándar: costo basado en peso total y distancia (usa una librería externa CorreoArgentina). Tiempo fijo de 5 a 7 días hábiles.

Envío express: costo calculado como porcentaje del valor total del pedido más un cargo base. Tiempo garantizado de 1 día hábil.

Retiro en sucursal: costo cero. Tiempo inmediato si hay stock en sucursal, o hasta 3 días si requiere traslado.

Además, el enunciado indica que nuevos métodos de envío podrían agregarse a futuro. Si implementáramos esto con condicionales dentro de Pedido o de un servicio de envío, estaríamos violando el principio Open/Closed (OCP), ya que cada nuevo método requeriría modificar el código existente. También se generaría un alto acoplamiento y una baja cohesión, porque la lógica de cada método quedaría mezclada.

### 2. Solución propuesta: Patrón Strategy

Aplicamos el patrón Strategy (GoF), que define una familia de algoritmos, los encapsula y los hace intercambiables. Esto permite que el algoritmo varíe independientemente del cliente que lo usa.

**La estructura es la siguiente:**

**Estrategia (Strategy):** interfaz MetodoEnvio que declara los métodos para calcular costo y tiempo de entrega.

**Estrategias concretas (ConcreteStrategy):** EnvioEstandar, EnvioExpress y RetiroSucursal, cada una con su propia implementación de los algoritmos.

**Contexto:** una clase ServicioEnvio (o directamente Pedido) que mantiene una referencia a la estrategia actual y delega en ella el cálculo.

Hemos optado por crear una clase ServicioEnvio separada, en lugar de cargar esta responsabilidad en Pedido, para:

Mantener el SRP (la clase Pedido ya gestiona su ciclo de vida con State y sus notificaciones con Observer).



Facilitar el cambio de estrategia en tiempo de ejecución, por ejemplo según selección del usuario o disponibilidad de stock.

Permitir que la lógica de envío sea reutilizable en otros contextos (ej. cotizaciones, presupuestos).

### **3. ¿Por qué esta decisión?**

State vs Strategy: State se usa cuando el comportamiento de un objeto cambia según su estado interno. Strategy se usa cuando el comportamiento cambia según un algoritmo seleccionado externamente. Aquí el método de envío es una elección del cliente o del sistema, no depende del estado del pedido, por lo que Strategy es el patrón adecuado.

Bajo acoplamiento: ServicioEnvio solo depende de la interfaz MetodoEnvio, no de las implementaciones concretas.

Alta cohesión: cada estrategia encapsula toda la lógica de un método de envío (cálculo de costo, tiempo, validaciones, etc.). La lógica no está dispersa en condicionales.

Escalabilidad: agregar un nuevo método (ej. "Envío internacional") solo requiere crear una nueva clase que implemente MetodoEnvio. No se modifica ServicioEnvio ni las estrategias existentes.

Testabilidad: cada estrategia se puede probar de forma aislada, mockeando las dependencias externas (como CorreoArgentina o EnvioExpress).

### **4. ¿Qué responsabilidad tiene cada componente?**

Componente Responsabilidad

MetodoEnvio (interfaz): Declara los métodos calcularCosto(pedido, direccion) y calcularTiempo(pedido, direccion).

EnvioEstandar: Implementa el costo usando peso total (calculado a partir de los Vendible del pedido) y distancia, invocando la librería CorreoArgentina. Tiempo fijo entre 5 y 7 días.

EnvioExpress: Implementa el costo como porcentaje fijo sobre el total del pedido más cargo base, usando EnvioExpress.calcularCosto. Tiempo fijo 1 día.

RetiroSucursal: Costo siempre 0. Tiempo: 0 si hay stock en la sucursal elegida, 3 días si requiere traslado (se consulta un servicio de stock).

ServicioEnvio (contexto): Almacena la estrategia actual y delega en ella los cálculos. Permite cambiar la estrategia en tiempo de ejecución.

## 5. Integración con el modelo existente

ServicioEnvio recibe un Pedido y una Direccion (y opcionalmente una Sucursal para retiro). Para calcular el peso total, recorre la lista de Vendible del pedido (cada Vendible podría tener un método getPeso(); si no lo tiene, se puede agregar como atributo dinámico o fijo).

Para EnvioExpress, necesita el valor total del pedido, que se obtiene con pedido.calcularTotal() (similar a lo que ya implementamos en el patrón Composite para paquetes).

La estrategia se asigna desde la capa de aplicación o UI, dependiendo de la selección del cliente.

## 6. Cumplimiento de principios SOLID

**SRP (Responsabilidad única):** ServicioEnvio se ocupa solo de delegar el cálculo; cada estrategia tiene una única razón de cambio (cambios en la lógica de ese método de envío).

**OCP (Abierto/Cerrado):** el sistema está abierto a agregar nuevas estrategias (cerrado a modificar ServicioEnvio y las existentes).

**LSP (Sustitución de Liskov):** cualquier estrategia concreta puede reemplazar a otra sin alterar el funcionamiento del contexto.

**ISP (Segregación de interfaz):** la interfaz MetodoEnvio declara sólo los métodos que el contexto necesita.

**DIP (Inversión de dependencias):** ServicioEnvio depende de la abstracción MetodoEnvio, no de implementaciones concretas.

# 4. Métodos de pago

## 1. Contexto

El sistema debe soportar múltiples medios de pago, donde cada uno requiere validar distintos datos y realizar diferentes operaciones, como reservar fondos y/o ejecutar la transacción a través de su propia API.

Sin embargo, el flujo general del procesamiento de un pago siempre sigue el mismo orden: validar los datos, reservar los fondos, ejecutar la transacción y notificar el resultado.

## 2. Solución propuesta: Patrón Template Method

El modelado de los métodos de pago fue diseñado con una estructura centrada en el Patrón de diseño **Template Method**, con este buscamos que el flujo de procesamiento esté definido en un único lugar, dejando que cada medio de pago implemente únicamente los pasos que le son propios.

Contamos con una clase abstracta (**MetodoPago<T extends> MedioDePago**) que define el contrato y el orden de ejecución de las validaciones.

- Define el método con el algoritmo que validará en el orden necesario todos las formas de pago.
- validarDatos(monto: Double, medio: T): void
- reservarFondos(monto: Double, medio: T): void
- ejecutarTransaccion(monto: Double, medio: T): void
- notificarResultado(monto: Double, medio: T): void (método definido por defecto).

De aquí se desprenden tres subclases concretas donde cada una está parametrizada con el tipo de medio que le corresponde.

**PagoPorTarjetaDeCredito<TarjetaDeCredito>** implementa los tres pasos delegando en `ApiTarjeta`, que recibe número de tarjeta, CVV y fecha de vencimiento.

**PagoTransferenciaBancaria<CuentaCorriente>** implementa los pasos delegando en `ApiTransferenciaBancaria`, que opera con CBU y alias.

**PagoPorBilleteraVirtual<BilleteraVirtual>** implementa los pasos delegando en `ApiBilleteraVirtual`, que trabaja con el monto y el saldo disponible.

Por su parte, MedioDePago encabeza la jerarquía de TarjetaDeCredito, CuentaCorriente y BilleteraVirtual. Cada una define y encapsula los datos propios y expone sus getters para que el método de pago correspondiente pueda acceder a ellos.

### **3. Asignación de responsabilidades**

Con todo esto buscamos cumplir los principios SOLID:

- Cada clase tiene una única responsabilidad.
- Podemos agregar nuevos medios de pago sin modificar los ya existentes.

### **4. Conclusiones**

El uso del patrón Template Method permite centralizar el flujo de procesamiento en un único lugar, evitando duplicación y garantizando que todos los medios de pago respeten el mismo orden de operaciones. La parametrización genérica de MetodoPago<T> resuelve el problema de que cada medio requiere datos distintos, manteniendo el polimorfismo sin perder seguridad de tipos. La separación entre la lógica de procesamiento y los datos del medio de pago contribuye a un diseño más mantenible y extensible, alineado con los principios SOLID.

# 5. Notificaciones del Pedido

## 1. Contexto

Cada vez que un pedido cambia de estado, distintos subsistemas deben reaccionar de forma independiente. Además, existe la posibilidad de que nuevos subsistemas podrían agregarse a futuro, sin que el módulo de pedidos los conozca. Por lo tanto, necesitamos una implementación que cumpla los siguientes requisitos:

- Permita que el pedido notifique que ocurrió un cambio de estado, informando el estado anterior y el nuevo.
- Desacople completamente el pedido de los subsistemas concretos (email, factura, fidelización).
- Sea extensible para agregar nuevos reactores sin modificar la clase Pedido.

Si implementamos esto con llamadas directas desde Pedido a cada subsistema, la clase quedaría acoplada a múltiples dependencias y sería muy costoso agregar o modificar reacciones.

## 2. Solución propuesta: Patrón Observer

Aplicamos el patrón Observer, que define una dependencia uno-a-muchos entre objetos: cuando el Pedido cambia de estado, notifica automáticamente a todos los observadores registrados.

La estructura es la siguiente:

Contexto (Pedido): Cada instancia de Pedido mantiene su propia lista de observadores. Cuando ese pedido en particular cambia de estado (en su método `setEstado`), itera sobre su propia lista y notifica únicamente a los observadores que fueron registrados en esa instancia. De este modo, los cambios en un pedido no afectan ni notifican a observadores de otros pedidos.

Observador (ObservadorPedido): interfaz que declara el método `onCambioEstado(evento, pedido)`.

Observadores concretos (NotificadorEmail, GeneradorFactura, ServicioFidelizacion): implementan la interfaz y definen su propia reacción ante el cambio de estado, basándose en el tipo de estado nuevo o anterior.

Evento (CambioEstadoEvento): transporta el estado anterior y el nuevo, para que cada observador pueda decidir si le interesa reaccionar y qué información necesita.

### 3. ¿Por qué esta decisión?

Bajo acoplamiento: Pedido no conoce las clases concretas de los subsistemas; solo depende de la interfaz ObservadorPedido. Esto permite cambiar, agregar o eliminar observadores sin modificar Pedido.

Alta cohesión: cada observador concentra su propia lógica de reacción. No hay código, condicionales o lógica relacionada a esto dentro de Pedido.

Escalabilidad: agregar un nuevo subsistema solo requiere crear una nueva clase que implemente ObservadorPedido y registrarla en el pedido. No se modifica la lógica de Pedido ni los observadores existentes.

### 4. Integración con el patrón State

State es responsable del comportamiento del pedido: qué operaciones son válidas, cómo transicionar y qué lógica de negocio lleva a cabo.

Observer se encarga de las reacciones externas que deben ocurrir como consecuencia de un cambio de estado.

Cada vez que un estado concreto ejecuta `pedido.setEstado(nuevoEstado)`, ese método:

- Cambia la referencia interna del estado.
- Crea un evento con el estado anterior y el nuevo.
- Notifica a todos los observadores registrados en ese pedido.

El cambio de estado es el punto de unión entre ambos patrones, sin mezclar las responsabilidades del uno y el otro.

## 5. ¿Qué responsabilidad tiene Pedido ahora?

Además de las responsabilidades ya descritas en el punto 2, Pedido ahora también:

- Contiene una lista de observadores (List<ObservadorPedido>) para el manejo de notificaciones.
- Expone el método agregarObservador(ObservadorPedido) para que se puedan registrar los suscriptores.
- En setEstado, notifica a todos los observadores con el evento correspondiente.

La lista de observadores es por instancia de pedido. Esto permite que cada pedido tenga sus propios suscriptores. No hay una lista global que afecte a todos los pedidos, lo que mantiene la flexibilidad.

## 6. Cumplimiento de principios SOLID

**Single Responsibility:** Pedido solo coordina la delegación y notificación; cada observador tiene una única razón de cambio.

**Open/Closed:** el sistema está abierto a agregar nuevos observadores y cerrado a modificar Pedido y los observadores existentes.

**Liskov substitution:** cualquier observador concreto puede reemplazar a otro sin alterar el funcionamiento de Pedido.

**Interface segregation:** la interfaz ObservadorPedido declara solo el método necesario (onCambioEstado).

**Dependency inversion:** Pedido depende de la abstracción ObservadorPedido, no de implementaciones concretas (email, factura, etc.).

## 6. Búsqueda en el catálogo

### 1. Contexto

Para realizar la búsqueda en el catálogo, el sistema debe soportar tanto criterios simples como criterios complejos, donde estos últimos se definen como combinaciones de criterios (ya sean simples o compuestos). Esto permite construir consultas más expresivas, incluyendo condiciones lógicas como AND, OR y NOT.

El cliente del sistema es el encargado de construir estos criterios, pudiendo combinar criterios simples y compuestos antes de ejecutar la búsqueda, lo que permite definir consultas de manera flexible y dinámica.

### 2. Propuesta de solución: Patrón Composite

La búsqueda por criterios fue diseñada utilizando el patrón de diseño Composite, ya que en el sistema contamos con criterios simples y complejos, donde los complejos son combinaciones de Criterios (tanto de criterios simples como de otros compuestos).

Se define una clase abstracta **Criterio** que establece el contrato común, con los siguientes métodos:

- `agregarCriterio(Criterio): void`
- `eliminarCriterio(Criterio): void`
- `validar(Vendible): boolean`

Los primeros dos métodos lanzan excepciones por defecto, indicando que no se pueden agregar o eliminar criterios. Esto se define así para todos los Criterios no compuestos. El método **validar** queda abstracto y será implementado de manera específica en cada subclase.

A partir de esta clase se desprende una subclase abstracta:

Por un lado, **CriterioCompuesto** que agrega una lista de Criterio donde almacenará los criterios que la compondrán. Además, redefine los métodos **agregarCriterio** y **eliminarCriterio**, permitiendo modificar la lista de Criterio definida.

De **CriterioCompuesto** se extienden dos clases concretas que son:

- **CriterioAND**: que implementa el método **validar(Vendible): boolean**, verificando que el Vendible cumpla con todos los criterios contenidos en la lista.
- **CriterioOR**: que implementa el método **validar(Vendible): boolean**, verificando que el Vendible cumpla al menos uno de los criterios contenidos en la lista.



Por otro lado, contamos con criterios simples que extienden directamente de la clase abstracta **Criterio**, cada uno encapsulando el dato necesario para realizar su validación.

De acá se desprenden tres clases concretas que conforman los criterios simples:

- **CriterioNombre**, donde se almacena un String que representa el texto a buscar. En la implementación de **validar(Vendible): boolean**, se verifica si dicho texto está contenido dentro del nombre del Vendible dado.
- **CriterioDescripcion**, donde se almacena un String que representa el texto a buscar. En la implementación de **validar(Vendible): boolean**, se verifica si dicho texto está contenido dentro de la descripción del Vendible dado.
- **CriterioPrecio**, donde se almacena un valor numérico que representa el precio máximo permitido. En la implementación de **validar(Vendible): boolean**, se verifica que el precio del Vendible sea menor o igual a dicho valor.

Por último nos quedó definido el **CriterioNOT** directamente de la clase abstracta **Criterio** ya que esta no tiene permitido agregar y quitar criterios. Este contiene un único **Criterio** (simple o compuesto) que utiliza para validar el Vendible recibido, invirtiendo el resultado booleano obtenido.

### 3. Asignación de responsabilidades

Con esta estructura se busca cumplir con los principios SOLID:

- Cada clase tiene una única responsabilidad:
  - Los criterios simples se encargan de validar una condición específica.
  - Los criterios compuestos se encargan de combinar múltiples criterios.
  - **CriterioNOT** se encarga de invertir el resultado de otro criterio.
- Se cumple con el principio de Open/Closed, ya que es posible agregar nuevos tipos de criterios, tanto simples como compuestos, sin modificar los ya existentes.
- Todos los criterios implementan la misma interfaz, permitiendo tratarlos de manera uniforme sin distinguir entre simples y compuestos.
- Las interfaces de las clases se mantienen acotadas y específicas, evitando responsabilidades innecesarias.

### 4. Conclusiones

El uso del patrón Composite permite construir estructuras de criterios complejas a partir de componentes simples, facilitando la composición de consultas dinámicas y reutilizables. Además, el diseño asegura bajo acoplamiento y alta cohesión, permitiendo extender el sistema de búsqueda de manera flexible sin impactar en las implementaciones existentes, manteniendo consistencia con los principios SOLID.

# 8. Reportes

## 1. Contexto

El sistema necesita generar un reporte de los productos más vendidos, ordenado por cantidad de unidades vendidas en un período determinado. Cada producto debe incluir el precio promedio, que puede diferir del precio base si se aplicaron descuentos (tanto individuales como de paquete).

La información de ventas se encuentra distribuida en los pedidos del sistema, y cada pedido contiene una lista de `ItemVendible`, que puede ser un `Producto` o un `Paquete`. Para generar el reporte, es necesario:

- Recorrer todos los pedidos del sistema.
- Filtrar solo los pedidos que estén en estado ENTREGADO (porque representan ventas reales).
- Procesar cada ítem, distinguiendo entre `Producto` y `Paquete` para extraer SKU, nombre, cantidad vendida y precio total.
- Acumular cantidades y precios por SKU.
- Calcular el precio promedio por unidad.
- Ordenar los resultados por cantidad de mayor a menor.

Además, el reporte debe poder exportarse en tres formatos diferentes: texto plano, CSV y HTML. El núcleo del reporte (recolección y estructuración de datos) debe ser independiente del formato de salida.

Si implementáramos esto con condicionales o modificando las clases `Producto` y `Paquete`, el código quedaría acoplado y difícil de extender. Cada nuevo reporte requeriría modificar las clases del catálogo, violando el principio Open/Closed (OCP).

## 2. Solución propuesta: Patrón Visitor

Aplicamos el patrón Visitor, que permite definir nuevas operaciones sobre una estructura de objetos sin modificar las clases de los elementos sobre los que opera.

La estructura es la siguiente:

- Visitor (`ReportVisitor`): interfaz que declara un método `visit` para cada tipo concreto (`Producto`, `Paquete`).

- Visitor concreto (ReporteProductosMasVendidos): implementa los métodos visit para acumular datos (cantidad y precio total) por SKU.
- Elemento visitable (ItemVendible): proporciona el método accept(ReportVisitor) que invoca al visitor según su tipo interno (Producto o Paquete).
- Cliente (Sistema): itera sobre los pedidos y llama a accept en cada ItemVendible, pasando el visitor.
- Resultado (ItemVenta): DTO que representa un producto en el reporte (SKU, nombre, cantidad, precio promedio).

El Visitor no modifica las clases Producto ni Paquete, sino que extrae la información necesaria a través de sus métodos públicos (getSku(), getNombre(), getPrecioFinal(), etc.).

## ¿Por qué esta solución?

- Separación de responsabilidades: Producto y Paquete no saben nada sobre reportes. La lógica de recolección vive en el Visitor.
- Extensibilidad: Se pueden agregar nuevos reportes si se desea, creando nuevos visitors sin modificar ItemVendible, Producto ni Paquete.
- Bajo acoplamiento: ItemVendible solo depende de la interfaz ReportVisitor, no de implementaciones concretas.
- Alta cohesión: Cada visitor concentra toda la lógica de un reporte específico.
- Cumple principio open/closed: El sistema está abierto a agregar nuevos visitors (cerrado a modificar ItemVendible y sus subtipos).
- Filtrado de estado: El cliente (Sistema) decide qué pedidos procesar (solo ENTREGADO), manteniendo al visitor ignorante de esa lógica.
- Independencia del formato: Los datos se recolectan en el visitor (List<ItemVenta>) y luego se exportan a diferentes formatos mediante un exportador separado (ExportadorReporte).